# Chapter 3.

# Constants, Variables, Data Type

# Character Set



## Characters in C grouped into 4 categories

- Letters
- Digits
- Special characters
- White spaces

# Character Set



| Letters |
| --- |
| Uppercase A…Z<br>Lowercase a…z |

| Digits |
| --- |
| All decimal digits 0…9 |

| White spaces | | |
| --- | --- | --- |
| Blank space | Horizontal tab | |
| Carriage return | New line | Form feed |

# Character Set



## Special Characters

, . ; ‘ “

| / \ ~ _

! ? $ % #

& ^ *

- + < >

( ) left/right parenthesis [ ] left/right bracket

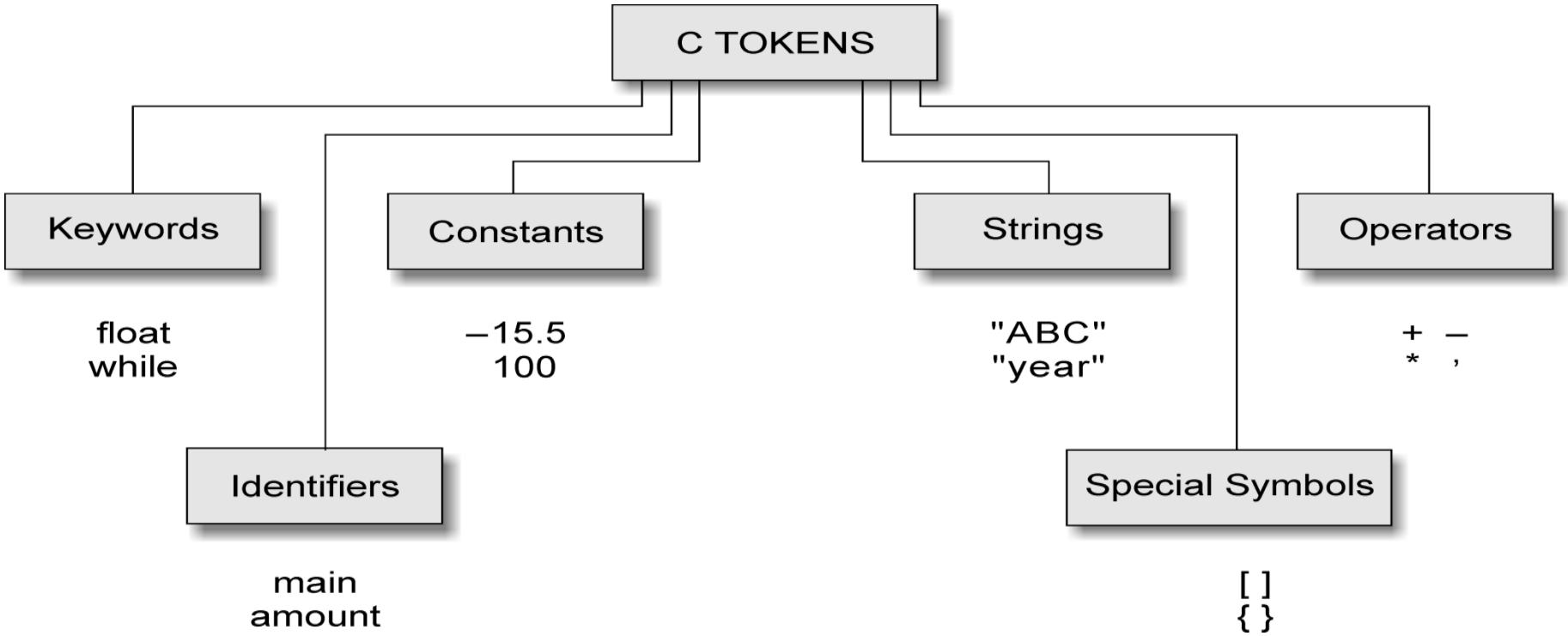
{ } left/right brace

# C Tokens



## Token ?

- In text: individual words and marks
- In a C program: the smallest individual units



## Keywords

- have fixed meanings that cannot be changed
- Must be written in lowercase

| Keywords | | | | |
|---|---|---|---|---|
| auto | do | goto | signed | unsigned |
| break | double | if | sizeof | void |
| case | else | int | static | volatile |
| char | enum | long | struct | while |
| const | extern | register | switch | |
| continue | float | return | typedef | |
| default | for | short | union | |

# Identifiers



- User defined names
  - names of variables, functions, arrays.
  - consist of letters and digits (LETTER FIRST: song123)
- Example:
  - int my_name;

```
/* my_name is an identifier used as a program variable*/
```

  - void CalculateTotal(int value)

```
/*
CalculateTotal is an identifier used as a function name
*/
```

## Rules for Identifiers

1. First Character must be an alphabet ( or _ )
2. Must consist of only letters, digits or underscore( _ ):
3. Only first 31 characters are significant
4. Cannot use a keywords
5. Must not contain white space
6. Case sensitive : Ss ≠ ss

| | |
|---|---|
| **Cannot** contain any arithmetic operators | R*S+T |
| … or any other punctuation marks… | #@x%!! |

# Constants



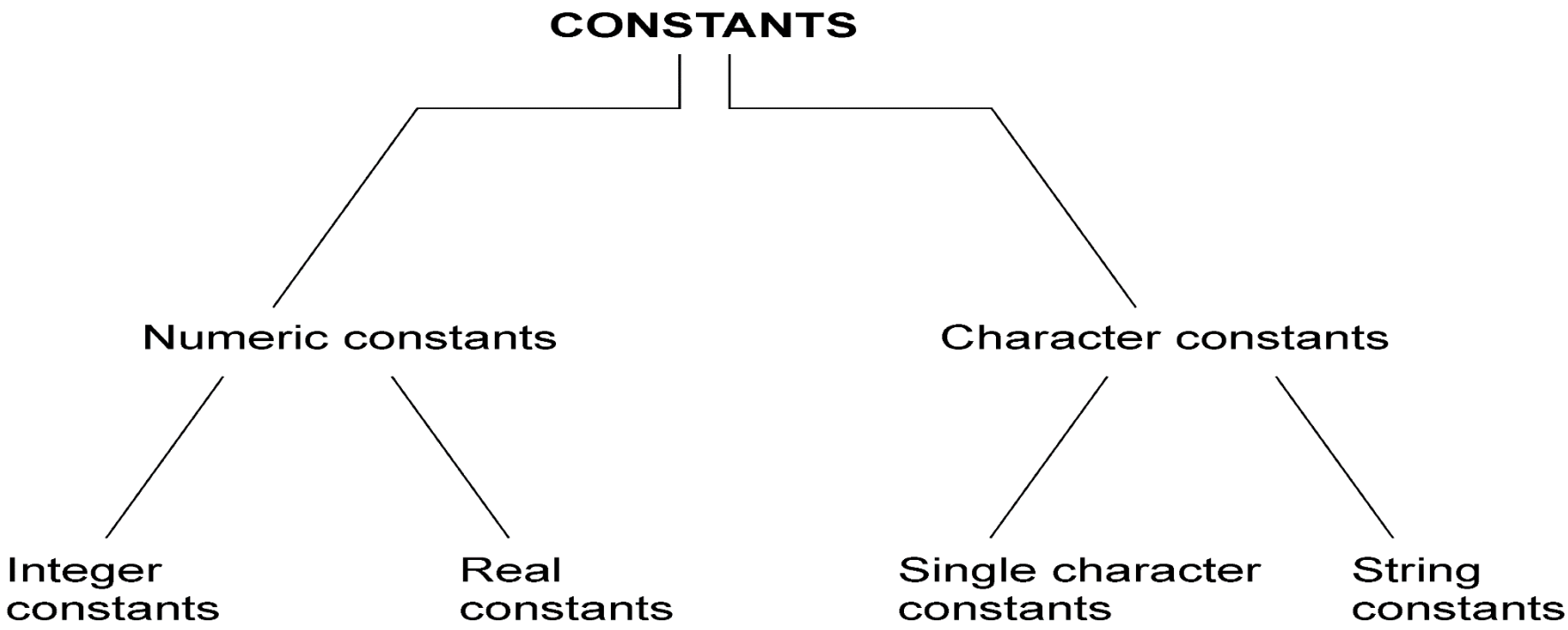



**Fig. 2.2** *Basic types of C constants*

# Constants

## Integer constants

- 3 types
  - ✓ **Decimal** : 0, 1, 2, …, 9, 321, -22, +99 **25,012 (Wrong)**
  - ✓ **Octal** (0~7, **leading 0**): **0**436, **0**, **0**71
  - ✓ **Hexadecimal** (0~9, A~F): **0X**2, **0x**9F, **0x**2af
- Largest integer value ?
  - ✓ Machine dependent
  - ✓ 16 bits, 32 bits, …

# Constants

## Integer constants

- Larger integer constants with qualifiers (U, L, UL)
  - ✓ 56789U (56789u) : unsigned integer
  - ✓ 987612347UL (ul) : unsigned long integer
  - ✓ 9876543L (l) : long integer

# Constants



**Program**

```
main()
{
    printf("Integer values\n\n");
    printf("%d %d %d\n", 32767,32767+1,32767+10);
    printf("\n");
    printf("Long integer values\n\n");
    printf("%ld %ld %ld\n", 32767L,32767L+1L,32767L+10L);
}
```

**Output**

```
Integer values
32767 -32768 -32759
Long integer values
32767 32768 32777
```

**Fig. 2.3** *Representation of integer constants on 16-bit machine*

# Constants

## Real constants

- Examples
  - ✓ 0.083 -0.75 435.36 +247.0
  - ✓ 215. .95 -.71 +.5 (+0.5)
- Exponential (or scientific) notation

  **Mantissa e exponent**

  - ✓ Valid: 0.65e4 12e-2 1.5e+5 3.18E3 -1.2e-1
  - ✓ Invalid: 1.5E+2.5 7.1e 4

# Constants

## Single character constants

- Examples
  - ✓ ‘5’ (!= 5) ‘X’ ‘;’ ‘ ’ (blank)
- Character constants have integer values

  ASCII code ➔ ASCII values
  - ✓ printf(“%d”, ‘a’); → 97
  - ✓ printf(“%c”, ‘97’); → a

  printf(“%c”, ‘S’); → S

# Constants

## String constant

- a sequence of characters within “ ”
  - ✓ “”Hello!” “1987” “5+3” “?...!” “x”
  - ✓ “X” ≠ ‘X’ : single character string ≠ single character

    No equivalent integer value assigned

- Chapter 8 deals with strings in detail !!!

# Constants

## Backslash character [Escape sequence]

| Escape Sequence | Effect |
|---|---|
| \a | Beep sound |
| \b | Backspace |
| \f | Formfeed (for printing) |
| \n | New line |
| \r | Carriage return |
| \t | Tab |
| \v | Vertical tab |
| \\ | Backslash |
| \” | “ sign |
| \o | Octal decimal |
| \x | Hexadecimal |
| \0 | NULL |

# Constants cont…

## Enumeration

- Values are given as a list
- Example:

```
enum Language { Korean, Malay, English, Arabic };
```

```
enum Language
        {
          Korean,
          Malay,
          English,
          Arabic
        };
```

# Variables



## Variable = data name used to store a data value

- **Data name** associated with a memory cell
  - whose value can change
- **Variable Declaration**: specifies the type of a variable
  - Example: `int num;`
- **Variable Definition**: ***assigning*** a value to the declared ***variable***
  - Example: `num = 5;`

# Variables



Variable = data name used to store a data value

| C basic data types | |
|---|---|
| int | Numbers |
| float | Single precision floating point |
| double | Double precision floating point<br>Long double |
| char | Characters |

real number

# Variables

## Variable declaration

- **Declare variables before they are used in a program**

```
int inches, feet, fathoms; /* int type: inches, feet, fathoms*/

float x, y;        /* float type: x, y */

char c;            /* character type: c */
```

# Variables

## Variable naming rule

1. Use a meaningful name for enhancing readability
2. Start with the alphabet or underscore ( _ )
3. Uppercase and lowercase are significant : Vmax ≠ vmax
4. ANSI standard recognizes a length of 31 characters
5. But, only 8 characters are significant (63 characters in C99)
6. It should not be a keyword
7. White space not allowed

# Variables

## Variable naming rule

**times10  get_next_char  _done  int_type**

**10times  get-next-char  char  Price$  Group one**

# Data Types



ANSI C supports three classes of data types:

- Primary (or Fundamental) data types
  - ✓ Integer (int)
  - ✓ Character (char)
  - ✓ Floating point (float)
  - ✓ Double-precision floating point (double), long double
  - ✓ void
- User-defined data types
- Derived data types
  - ✓ Arrays, functions, structures, pointers

# Data Types

## Primary data types

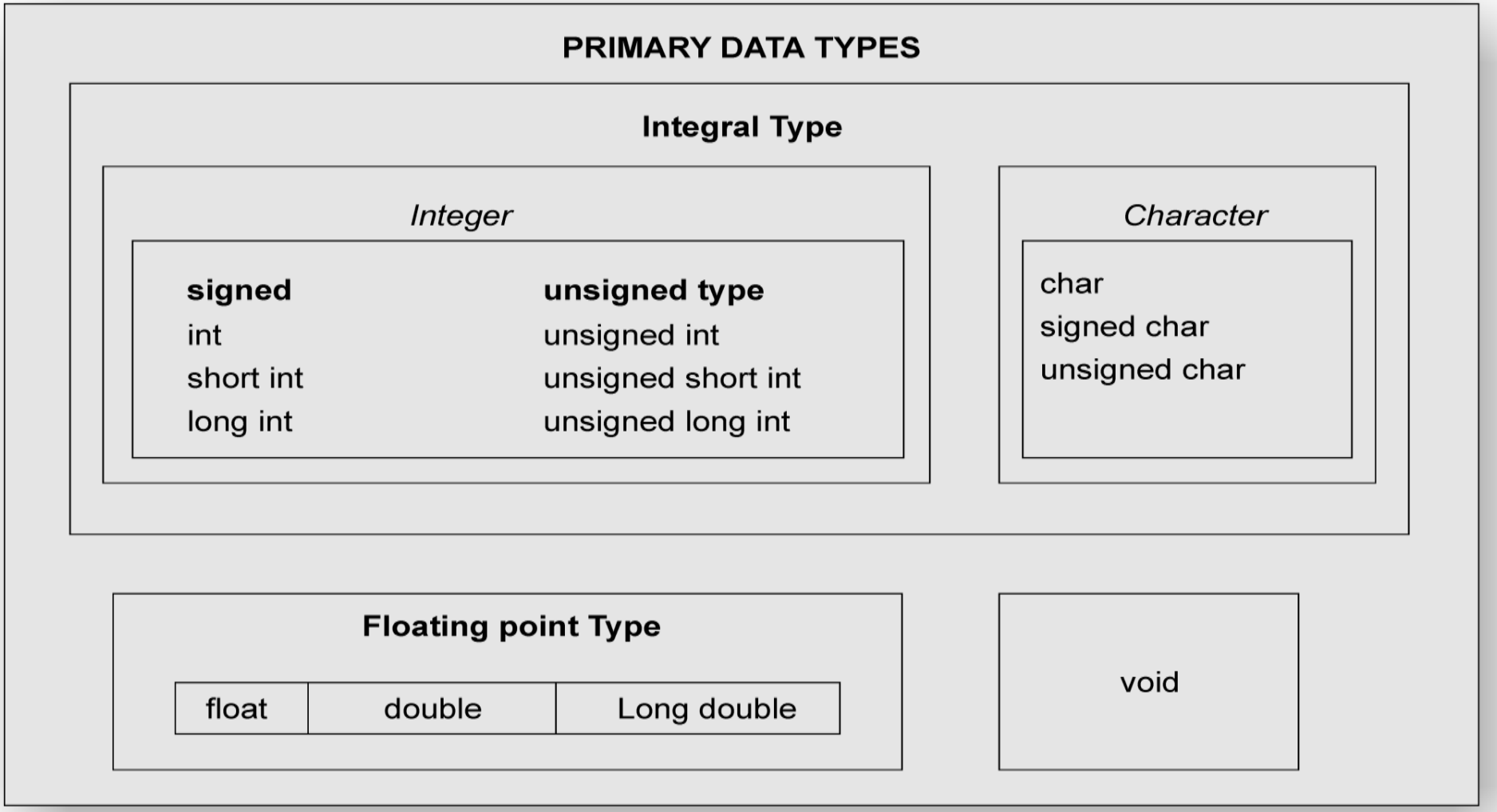



**Fig. 2.4** *Primary data types in C*